B.Tech Computer Science and Engineering
Final Year Project - Group 2

Project Title : Agentic AI for Code Migration and Equivalence Verification

1. Introduction and Overview

Modern software engineering frequently requires upgrading legacy codebases whether transitioning between major language versions (like Python 2 to Python 3) or translating an application from one language to another (such as Python to Java). While this migration is necessary for security and performance, doing it manually is slow, expensive, and prone to subtle bugs.

Traditional rule-based transpilers struggle with runtime nuances and produce unidiomatic code. At the same time, asking frontier Large Language Models (LLMs) to translate entire repositories in a single prompt consistently fails due to context limits, lost dependencies, and ungrounded hallucinations.

This project presents a multi-agent orchestration and verification pipeline designed to make automated code migration reliable and provably correct. Instead of attempting brittle, full-repository translations at once, the system breaks the process into a structured, symbolic workflow:

1. Static Decomposition Before Generation: A symbolic static analysis layer parses repository dependencies and decomposes the code into small, isolated fragments. These fragments are processed in reverse call order, ensuring the LLM only translates manageable units with complete context.
2. Targeted Equivalence Verification: Rather than relying on incomplete legacy unit tests, a dedicated testing agent performs control-flow and data-flow analysis to generate targeted differential tests. The legacy and translated fragments are run on identical inputs in a sandboxed execution environment to verify matching runtime behavior.
3. Structured Repair Loops: When a test fails, the system avoids generating code from scratch. Instead, it sends a structured failure report including assertion

diffs, stack traces, and actionable error context to a specialized repair agent for precise patching.

By combining the structural guarantees of static analysis with the flexibility of multi-agent LLM systems and empirical sandbox execution, the project transforms code migration from a risky, probabilistic guess into an automated, verified engineering process.

2. Literature Review

In *TransAgent* (Yuan et al., 2026), the authors address the systemic failures in repository-wide debugging that occur during LLM-driven code translation. Their research highlights how models frequently fail to resolve ambiguous compiler diagnostic messages and high-level semantic discrepancies. Utilizing a novel benchmark of contemporary competitive programming tasks across Python, Java, and C++, they implement a sophisticated four-agent orchestration layer. This pipeline integrating specialized agents for initial translation, syntax rectification, control-flow alignment, and semantic repair was validated using models such as GPT-4o and Claude 3.5 Sonnet. The methodology leverages symbolic control-flow decomposition to establish fine-grained mapping between source and target fragments. By executing these blocks in parallel to identify intermediate state divergence, the framework enables localized, precise patching, ultimately yielding a 33.3% performance gain over existing baselines like UniTrans.

In *ReCodeAgent* (Ibrahimzada et al., 2026), the researchers confront the critical lack of automated, language-independent frameworks for large-scale code migration. Their work identifies the limitations of previous neuro-symbolic models, which necessitated extensive manual engineering for specific language pairs, and single-agent LLM systems that often succumbed to erratic execution paths. Utilizing an extensive evaluation suite of 118 industrial-grade repositories across six programming languages including C to C++, Java to Python, and TypeScript to JavaScript the study proposes a robust four-agent orchestration pipeline. By integrating specialized agents for analysis, planning, generation, and validation through the Model Context Protocol (MCP), the system effectively maps dependencies and conducts dynamic runtime verification. The methodology achieves a significant 60.8% enhancement in test pass rates over state-of-the-art neuro-symbolic baselines. Furthermore, ablation studies indicate that removing the

multi-agent coordination leads to a 40.4% decline in functional equivalence and substantially increased repair latency.

In *MatchFixAgent* (Ibrahimzada et al., 2026), the researchers investigate the persistent challenges of erroneous equivalence assertions and deficient bug remediation within automated code translation pipelines. Their study highlights how significant language-specific engineering requirements and a dependence on substandard legacy test suites undermine reliability. Utilizing a robust benchmark of 2,219 function pairs extracted from 24 industrial-grade repositories encompassing over 900,000 lines of code, the work introduces a language-agnostic multi-agent orchestration framework. This system integrates specialized agents for deep semantic analysis of control-flow, data-flow, and I/O contracts alongside dedicated units for test generation, repair, and final verdict assessment. The methodology demonstrated exceptional precision, providing definitive (in)equivalence verdicts for 99.2% of units and achieving a 50.6% repair rate for identified defects—substantially outperforming the 18.5% success rate of prior state-of-the-art baselines.

In *RepoTransAgent* (Guan et al., 2025), the researchers investigate the inherent limitations of repository-aware code translation within contemporary LLM systems. Their work identifies that standard models frequently falter on intricate codebases due to restricted cross-file semantic comprehension and static prompt architectures. By utilizing a benchmark of Java-to-C# migration tasks derived from prominent industrial repositories like Lucene and JGit, the study introduces a sophisticated multi-agent orchestration pipeline. This framework integrates Retrieval-Augmented Generation (RAG) to resolve complex dependency graphs and implements a dynamic reflection-driven repair loop based on execution feedback. Validated using GPT-4o and DeepSeek, the methodology achieved substantial improvements in performance, elevating compilation success rates from 30.47% to 55.34% and functional test pass rates from 28.16% to 45.84% compared to state-of-the-art baselines.

3. Challenges Identified

Recent advancements in automated code migration have transitioned the field from single-shot LLM translation to structured, multi-agent frameworks. However, a systematic synthesis of the literature reveals critical bottlenecks that current

systems either overlook or resolve sub-optimally. This project directly addresses the following core challenges:

- **The "Syntax-Correct, Functionally Broken" Gap:** Techniques like AlphaTrans achieve upwards of 99% syntactic compilation by decomposing repositories via static call graphs, yet their functional pass rates plateau below 30%. Existing systems treat successful compilation as an indicator of progress, but without deep semantic alignment, translated code frequently hides critical logical discrepancies, silent edge-case failures, and unhandled runtime exceptions.
- **Lack of Standardized Tooling and Deterministic Execution:** Existing translation pipelines require heavy manual engineering for each language pair's static analyzer or rely on ad-hoc shell execution within custom environments. This lack of standardized communication between symbolic tooling (AST parsers, call-graph extractors) and neural agents creates brittle integration layers and prevents clean tool reuse across different migration tasks.
- **Prohibitive Costs and Closed-Ecosystem Lock-in of Proprietary LLMs:** Most existing agentic frameworks default to expensive, proprietary frontier APIs (e.g., GPT-4o, Claude 3.5 Sonnet). While effective, multi-agent validation loops require dense iterations across hundreds of code fragments, driving up operational costs to unsustainable levels for large repositories and preventing local, on-premise execution for privacy-sensitive enterprise codebases. Existing work has not fully explored cost-optimized tiered architectures that effectively delegate heavy synthesis to capable open-weight coding models (e.g., DeepSeek-Coder, Qwen2.5-Coder) while using lightweight models for orchestration glue.

4. Project Plan and Architecture

Architecture :-

Layer 0: Static Analysis and Neuro-Symbolic Decomposition (Symbolic Layer)
Before any LLM invocation, deterministic AST parsers (LibCST for Python, tree-sitter for multi-language ASTs) process the legacy repository. This layer extracts the full dependency call graph, maps inter-module references, and establishes a bottom-up **reverse call order** (processing leaf functions before

callers). It also generates a skeleton scaffold in the target language to anchor future fragments.

Layer 1: Deterministic Orchestration Engine
Implemented via LangGraph, the orchestrator replaces unconstrained conversational agent routing with a finite-state machine. Every state transition (Spec Extraction to Translation to Test Synthesis to Execution to Repair) is explicitly typed, with strict retry bounds (maximum 3 iterations per unit) and a Human-in-the-Loop (HITL) escalation gate for unresolvable semantic ambiguities.

Layer 2: Multi-Agent Execution & Verification Pipeline

- **Analyzer Agent (Lightweight LLM):** Inspects the target code fragment, its incoming parameters, and upstream dependencies to generate a structured contract (type signatures, expected I/O boundaries, and migration hazard flags such as Python 2 str/unicode conversions).
- **Translator Agent (Open-Source Heavy Coding Model):** Synthesizes the target fragment directly into the skeleton project using the structured specification.
- **Test-Generation Agent:** Uses data-flow and control-flow boundary analysis to synthesize fine-grained differential unit tests targeting edge cases, type boundary limits, and state mutations.
- **Executor & Sandbox (Docker Harness via MCP):** Executes the legacy and translated fragments in parallel containers on identical inputs, streaming runtime variable states, standard output, and execution traces.
- **Verdict Agent:** Compares runtime behaviors. If execution outputs or side effects diverge, it outputs an Inequivalent verdict along with a structured failure record.
- **Repair Agent:** Consumes the structured failure record (failing inputs, stack traces, variable state diffs, and exact localized blocks) to apply surgical patches to the translated fragment.

Model Context Protocol (MCP) Integration:
All tool interactions querying AST dependencies from Layer 0 or executing differential tests inside Docker containers are exposed to agents via standardized MCP servers. This eliminates ad-hoc bash generation by models, ensuring safe, deterministic JSON-RPC interactions.

Phase	Milestone / Deliverable	Core Tasks & Engineering Scope
Phase 1: Tooling & Sandbox Setup	Layer 0 Parsers & Docker Testbed	Implement LibCST/tree-sitter call-graph extractors; containerize execution harness based on RepoTransBench/AlphaTrans artifacts; configure MCP server wrappers.
Phase 2: Agent Graph Implementation	LangGraph Pipeline & Tiered Models	Construct deterministic LangGraph state machine; deploy local inference endpoints for **Qwen2.5-Coder / DeepSeek-Coder** (heavy translation/repair) and smaller models (orchestration/analyzer); establish structured JSON schema exchange.
Phase 3: Targeted Verification Loop	Differential Testing Engine	Implement the Test-Generation and Verdict agents; enable parallel execution trace capture and variable diffing inside the Docker sandbox.
Phase 4: Benchmarking & Evaluation	Empirical Validation & Ablations	Benchmark against Python 2 to 3 hazard suites and Python to Java subsets from RepoTransBench; run model-choice ablations (open-weight tiers vs. proprietary baselines) measuring token cost, compilation rate, and semantic equivalence.

5. Training and Experiment Plan

This empirical evaluation focuses on demonstrating that combining symbolic decomposition with a targeted verification loop achieves high semantic equivalence while optimizing compute and inference costs via open-weight model tiers.

Scope & Benchmark Datasets

Experiments will target two core migration regimes, moving from narrow, hazard-dense modernization to cross-paradigm translation:

Same-Language Modernization (Python 2 to Python 3)

- Target Hazard: Silent data-corruption risks, primarily the string/bytes separation hazard (str/unicode to bytes/str), integer division semantics, and iterator/generator shifts.
- Dataset: Curated repositories and legacy packages from Python 2 migration benchmarks containing both legacy unit tests and unexercised edge cases.

Small-Scale Cross-Language Translation (Python to Java)

- Target Challenge: Dynamic-to-static typing impedance mismatches, class/interface restructuring, and standard library API mapping.
- Dataset: Compact, multi-file repository units sourced directly from RepoTransBench (subsets with self-contained dependencies and runnable Docker environments).

Model Tiering & Cost-Optimization Strategy

Rather than relying universally on expensive, closed frontier APIs, the pipeline employs a tiered, task-specialized model allocation

Agent Role	Workload Characteristics	Target Backbone Models	Rationale / Cost Profile
Translator Agent	High-complexity syntax & idiom generation	DeepSeek-Coder-V2 / Qwen2.5-Coder (32B/7B), DeepSeek-R1	Maximizes code synthesis quality; deployed locally or via cost-effective open-weight endpoints.

Repair Agent	Precise, localized block patching from diffs	DeepSeek-Coder / Qwen2.5-Coder / DeepSeek-R1	High reasoning capability required to interpret variable diffs and trace mismatches.
Test Generation Agent	Edge-case boundary synthesis from CFG/DFG specs	Qwen2.5-Coder-7B / Llama-3.1-8B-Instruct	Standard structured generation targeting explicit parameter and path boundaries.
Analyzer & Verdict Agents	JSON extraction, contract mapping, report validation	Llama-3.1-8B / Qwen2.5-7B / Claude-3.5-Haiku	Lightweight, high-throughput tasks serving as orchestration glue.

Evaluation Metrics & Baselines

Performance will be evaluated on strict dynamic execution and semantic fidelity rather than superficial token-matching metrics:

Primary Metrics:

- Syntactic Compilation Rate (%): Percentage of translated repository fragments that parse and compile cleanly.
- Legacy Test Pass Rate (%): Pass rate when executing existing repository unit tests.
- Differential Equivalence Rate (%): Percentage of fragments passing 100% of newly synthesized, fine-grained boundary tests running against the legacy source.
- Repair Convergence Efficiency: Mean iterations and token budget consumed to resolve detected semantic discrepancies (capped at 3 retry loops).
- Cost per Repository ($ / repo): Financial and token expenditure across the full migration trajectory.

Baselines for Comparison:

- Rule-based Transpilers: 2to3, libCST-based codemods.
- Single-Shot / Naive LLM Translation: Zero-shot and few-shot repository-level translation using GPT-4o and DeepSeek-Coder.
- Prior SOTA Multi-Agent Frameworks: TransAgent, AlphaTrans, and ReCodeAgent baselines on identical RepoTransBench instances.

6. References

- Yuan, Zhiqiang, et al. "TransAgent: Enhancing LLM-based code translation via fine-grained execution alignment." *Proceedings of the ACM on Software Engineering* 3.FSE (2026): 1582-1604.
- Wang, Yanli, et al. "RepoTransBench: A Real-World Multilingual Benchmark for Repository-Level Code Translation." *arXiv preprint arXiv:2412.17744* (2024).
- Guan, Ziqi, et al. "Repotransagent: Multi-agent llm framework for repository-aware code translation." *arXiv preprint arXiv:2508.17720* (2025).
- Ibrahimzada, Ali Reza, et al. "ReCodeAgent: A Multi-Agent Workflow for Language-agnostic Translation and Validation of Large-scale Repositories." *arXiv preprint arXiv:2604.07341* (2026).
- Ibrahimzada, Ali Reza, et al. "Alphatrans: A neuro-symbolic compositional approach for repository-level code translation and validation." *Proceedings of the ACM on Software Engineering* 2.FSE (2025): 2454-2476.
- Ibrahimzada, Ali Reza, et al. "Matchfixagent: Language-agnostic autonomous repository-level code translation validation and repair." *arXiv preprint arXiv:2509.16187* (2025).
- Ou, Guangsheng, et al. "RustRepoTrans: Repository-level Code Translation Benchmark Targeting Rust." *arXiv preprint arXiv:2411.13990* (2024).
- Ou, Guangsheng, et al. "Evolving Triple Knowledge-Augmented LLMs for Code Translation in Repository Context." *arXiv preprint arXiv:2503.18305* (2025).
- Valenzuela Ramírez, Andrea, et al. "CRusty: Enhancing LLMs for C-to-Rust transpilation." (2025): 153-153.
- Xue, Pengyu, et al. "TransLibEval: Demystify Large Language Models' Capability in Third-Party Library-Targeted Code Translation." *Proceedings of the ACM on Software Engineering* 3.FSE (2026): 920-942.

Suggested Outline of Presentation :

1. Introduction - Application Domain
2. Motivation
3. Challenges
4. Problem Statement, Goal, and Outcomes
5. Literature Survey
6. Project Block or flow diagram.
7. Data set details and Implementation details, if done
8. Timeline
9. Plan for next 1 month till 2nd Evaluation